Face Attendance System - Technical Deep Dive

1. System Architecture & Workflow

High-level Data Pipeline

The application operates as a standalone desktop kiosk rather than a distributed web architecture.

- Webcam Frame Ingestion: Frames are captured locally via OpenCV (cv2.VideoCapture).
- Processing: Frames are converted to grayscale and passed to a Haar Cascade classifier for face detection.
- Recognition: Detected face regions are cropped, resized to 200x200, and passed to a local LBPH (Local Binary Patterns Histograms) model.
- Database Persistence: Identified users are logged into a local SQLite database (attendance.db). The system checks for duplicate daily punches before committing.

Flow Diagram / Sequence

Registration Flow (Admin Mode):

1. Admin enters User ID, Name, and Date.
2. System captures 40 grayscale face samples using the webcam, saving them to dataset/<user_id>/.
3. System reads all images in dataset/ and trains the LBPHFaceRecognizer.
4. The trained model is saved as models/trained_model.yml.

Attendance Verification Flow (Kiosk Mode):

1. User stands in front of the camera; 'Start Attendance' is triggered.
2. OpenCV detects the face bounding box.
3. LBPH predicts the face ID and provides a 'confidence' score (lower is better, acting as a distance metric).
4. If confidence < 70, the user ID is queried in the database.
5. mark_attendance() attempts an insert. If a unique constraint fails (User + Date), it flags as 'Already Marked'.

2. Core ML & Computer Vision Components

(Note: The prompt mentions advanced components like ArcFace, FAISS, and Liveness detection. The current codebase implements a simpler approach. Comparisons are provided below.)

Feature Extraction & Embedding

- Implemented Model: Haar Cascades for detection + LBPH (Local Binary Patterns Histograms) for recognition.
- Why it was chosen: LBPH is computationally inexpensive, runs seamlessly on CPUs without GPU acceleration, and works well for small, localized datasets.
- Comparison to ArcFace/dlib: ArcFace extracts 128D/512D float embeddings using deep CNNs (like ResNet). LBPH relies on micro-patterns (texture) rather than deep semantic embeddings.
- Raw Images: In a Deep Learning pipeline, raw images are discarded once embeddings are generated. However, in

this LBPH implementation, raw grayscale images *are* stored locally in the dataset/ folder because LBPH requires full retraining on the dataset whenever a new user is added.

Distance Metrics & Search

- Implemented Logic: The system uses LBPH's built-in prediction, which calculates the Chi-Square or Euclidean distance between the histogram of the input face and the trained histograms.
- Comparison to FAISS: FAISS is NOT used here. FAISS provides O(1) / sub-millisecond vector indexing for millions of 512D embeddings. Since this system uses LBPH, the model handles the matching internally. SQLite is only used to map the predicted integer ID to a user's name.

Anti-Spoofing & Liveness

- Implemented Logic: The current codebase does not implement active liveness detection.
- Future Recommendation: To prevent 2D presentation attacks (holding up a photo), mechanisms like EAR (Eye Aspect Ratio via landmarks) or texture analysis (LBP-SVM) should be integrated. Currently, a printed photo could trick the Haar + LBPH pipeline.

Face Attendance System - Technical Deep Dive

3. Backend & Database Architecture

Framework Choice

- Implemented Framework: CustomTkinter for the desktop GUI.

- Comparison to FastAPI/Django: The system is a Desktop Application, not a Web API. CustomTkinter provides a bundled, zero-latency local interface (Kiosk) without needing a local web server or browsers.

Database Design

- Database Used: Raw SQLite3 via the built-in module (No SQLAlchemy ORM).

- Schema:

  * admins: username (PK), password_hash

  * users: id (PK), user_code (Unique), name, registered_date

  * attendance: entry_id (PK), user_code, name, date, time (UNIQUE constraint on user_code + date)

- Duplicate Prevention: Handled at the DB level. INSERT INTO attendance fails gracefully via sqlite3.IntegrityError if the user has already punched in for the current date.

- SQL Injection: Parameterized queries are used (e.g., execute('... WHERE id = ?', (id,))), preventing SQL injection.

4. File-by-File Technical Breakdown

- app.py: Main application entry point. Handles UI (CustomTkinter), camera integration, and state logic (Kiosk vs Admin).

- db_manager.py: Handles all SQLite database interactions, schema initialization, and data querying.

- model_trainer.py: Handles webcam data collection (dataset generation) and trains the LBPH model.

- generate_pdf.py: Standalone script to generate PDF reports of the workflow using FPDF.

- requirements.txt: Lists Python dependencies required to run the project.

- attendance.db: Local SQLite database file containing persistent application state.

- haarcascade_*.xml: Pre-trained model weights for OpenCV's face detector.

5. Production Readiness & Edge Cases

Handled Edge Cases

- Duplicate Attendance: Database-level UNIQUE constraint prevents a user from marking attendance multiple times a day.

- Missing Models: app.py gracefully degrades, showing a 'No Model Loaded' banner if trained_model.yml is missing.

- Date Standardization: parse_and_standardize_date handles multiple date formats to ensure DB consistency.

Limitations & Scalability (Areas for Improvement)

- Lighting/Pose Variance: Haar Cascades are limited to frontal faces and require good ambient lighting. LBPH struggles with extreme lighting changes.

- Scalability: Retraining the entire LBPH model from scratch via model_trainer.py becomes computationally expensive as the dataset grows beyond a few hundred users.

- Threshold Calibration: The confidence threshold is hardcoded to < 70. This might need dynamic calibration to balance False Positives and False Negatives.